JOMHACK
EMPOWERING INNOVATION

```
> bootcamp.learn()
```

JOMHACK
EMPOWERING INNOVATION

JomCode Coding Bootcamp

CHAPTER 02

Git and GitHub workflow

```
follow / verify / explain
```

JOMHACK
EMPOWERING INNOVATION

The workflow and prerequisites

01 Git tracks local history. GitHub stores a remote copy and supports collaboration.

02 Complete Chapter 1 first. Use a practice folder containing hello.py.

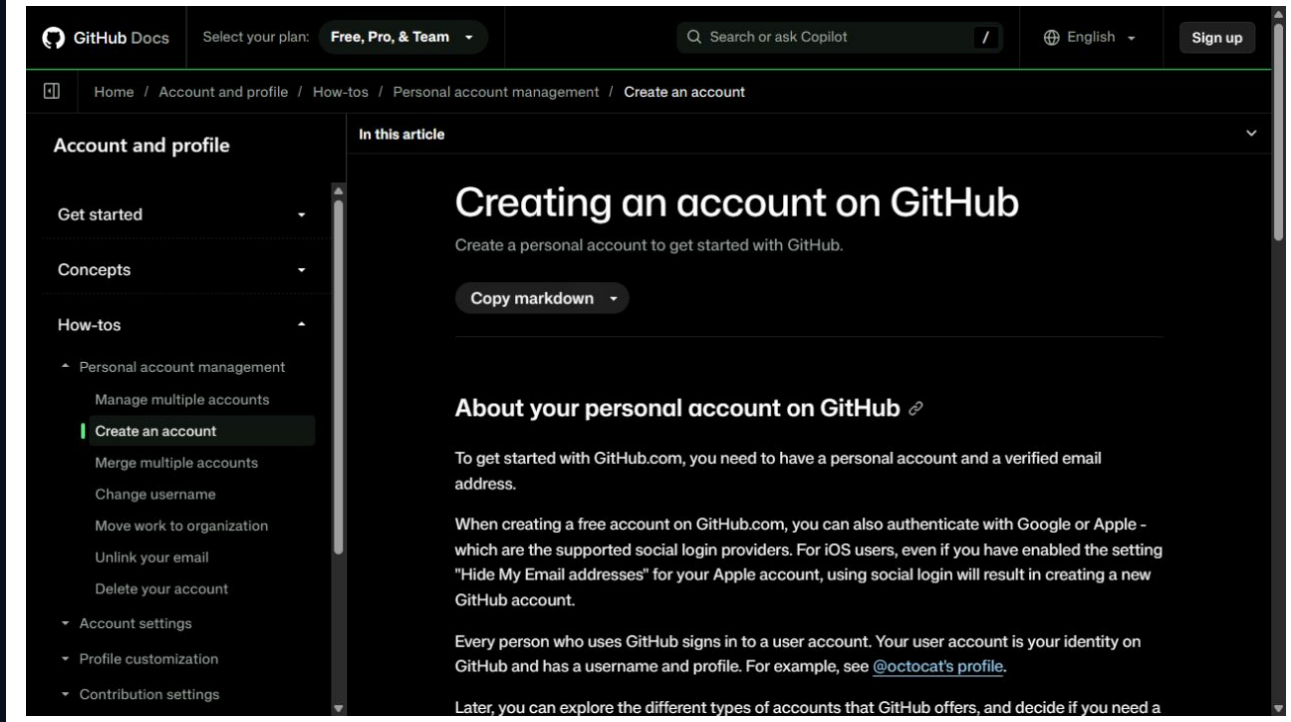
03 Core workflow: edit, inspect, stage, commit, push, then verify on GitHub.

04 The final section covers branches and pull requests as advanced reference.

JOMHACK
EMPOWERING INNOVATION

GitHub account

1. Open github.com and select Sign up.
2. Enter your own email, password and username. Complete any verification yourself.
3. Verify your email through the message GitHub sends.
4. Sign in and keep recovery information private. Enable two-factor authentication using GitHub account settings.



Official screen reference; interface labels may change.

Official reference

New repository

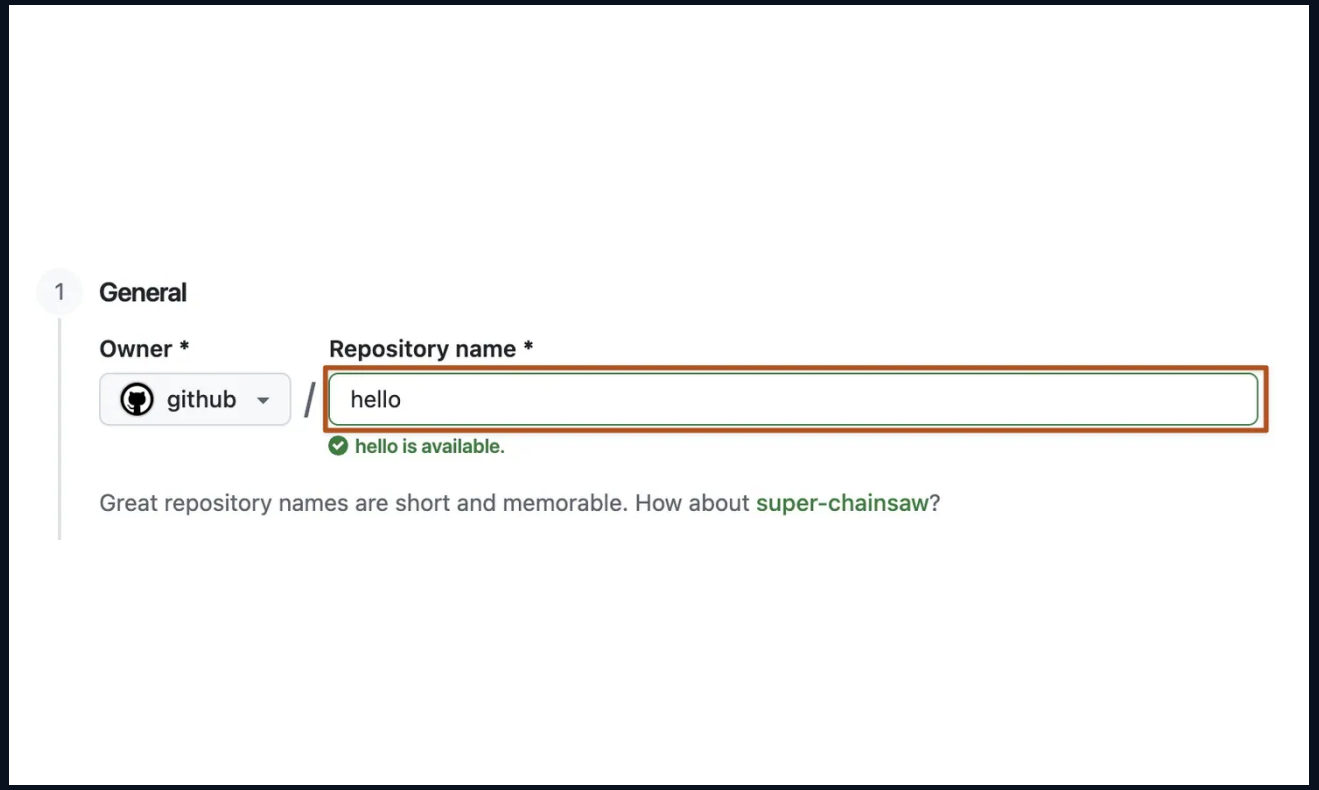
1. In GitHub, use the plus menu at the top right, then New repository.

2. Set the owner to your account. Name the practice repository jomcode-practice.

3. Choose visibility deliberately: Public is visible to everyone; Private restricts repository access.

4. For this local-first workflow, leave README, .gitignore and license initialization unchecked.

5. Select Create repository. Keep the resulting Quick setup page open.



Official screen reference; interface labels may change.

Official reference

Repository URL and local folder

1. On Quick setup, select HTTPS and copy the repository URL.

2. In VS Code, open the existing jomcode-practice folder.

3. Open Terminal > New Terminal and confirm hello.py is present.

4. YOUR_REPOSITORY_URL below means the full URL you copied, ending in .git.

This practice repository URL is separate from the Chapter 28 guided-project URL, which remains blank.

```
git --version
pwd
```

JOMHACK
EMPOWERING INNOVATION

Commit identity

1. Set your commit display name and a verified GitHub email or GitHub-provided noreply email.

2. Run the read-back commands to confirm both values.

3. Commit identity records authorship. It does not authenticate a push.

4. These global settings apply to future repositories on this computer.

```
git config --global user.name "YOUR NAME"
git config --global user.email "YOUR EMAIL"
git config --get user.name
git config --get user.email
```

Official reference

Files to exclude before the first commit

1. Create a file named .gitignore in the practice folder.
2. Add the patterns below and save.
3. Keep example environment templates with placeholders. Never stage populated environment files.
4. Ignoring a file does not remove it if it is already tracked. Check before your first commit.

```
.env
.env.*
!.env.example
.venv/
node_modules/
__pycache__/
dist/
```

First local commit

1. Initialize Git in the practice folder.
2. Inspect status. Stage the intended files explicitly.
3. Review the staged diff. Confirm no keys or environment values appear.
4. Create a commit with a short message describing the change.

Expected result: Git reports a commit identifier and the files included.

```
git init
git status
git add hello.py .gitignore
git diff --staged
git commit -m "Add first Python example"
git branch -M main
```

Connect and push

1. Replace YOUR_REPOSITORY_URL with the copied HTTPS URL.

2. Add origin, inspect it, then push main.

3. If Git opens a browser authentication flow, sign in to your GitHub account.

4. GitHub does not accept your normal account password as an HTTPS Git password. Use the supported browser/credential-manager flow.

5. Refresh the repository page. Verify hello.py, .gitignore and your commit message.

```
git remote add origin YOUR_REPOSITORY_URL
git remote -v
git push -u origin main
```

Official reference

Repository verification

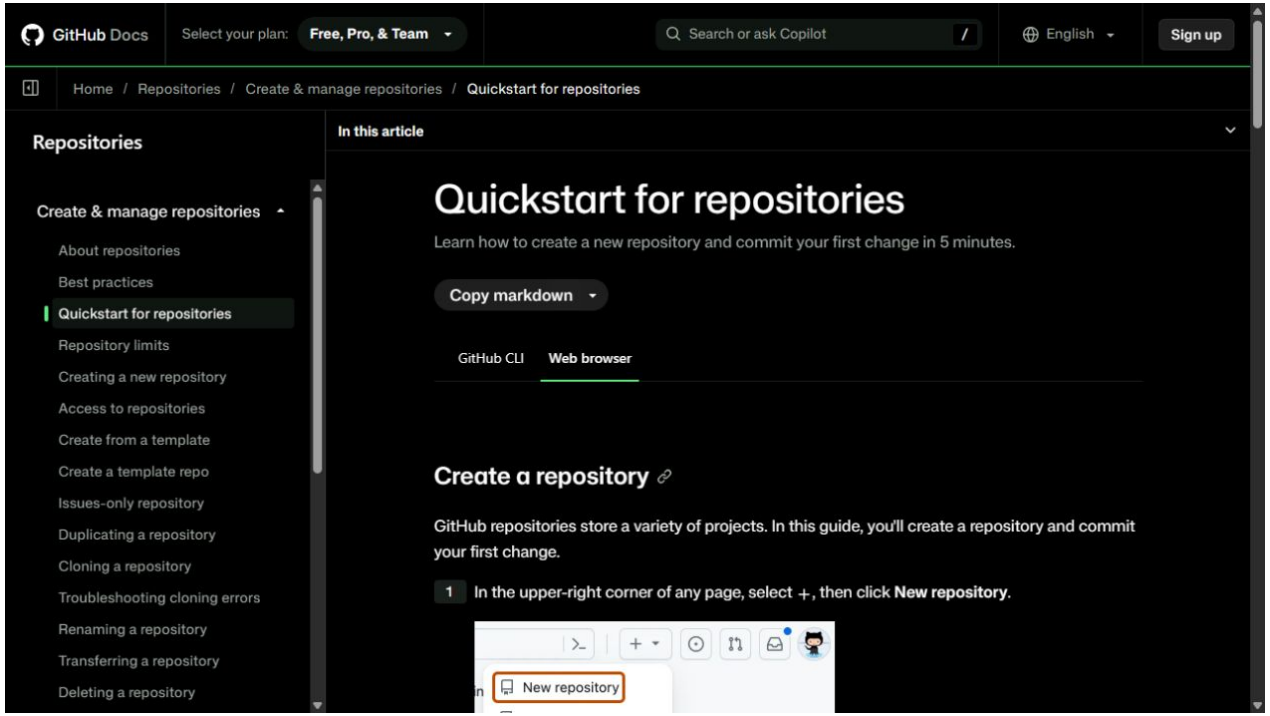
On the repository Code tab, confirm the selected branch is main.

Open hello.py and compare it with the saved local file.

Open commit history and find Add first Python example.

Confirm .env and generated dependency folders are absent.

A successful local commit does not prove the push succeeded; verify the remote page.



Official screen reference; interface labels may change.

Official reference

Update the repository

1. Change the greeting in hello.py and save.
2. Run it locally and inspect the diff.
3. Stage the changed file, review the staged diff, commit and push.
4. Refresh GitHub and verify the new greeting and second commit.

```
git status
git diff
git add hello.py
git diff --staged
git commit -m "Update greeting"
git push
```

A remote change and git pull

If the remote branch changed elsewhere, keep your local working tree clean before pulling.

Run git pull --ff-only to accept a straightforward update without silently creating a merge commit.

If histories diverge or a conflict appears, inspect git status and ask the instructor to review the conflicting changes.

Avoid force-pushing as a first response to a rejected push.

```
git status
git pull --ff-only
git log --oneline -5
```

JOMHACK
EMPOWERING INNOVATION

Common problems

01 Not a git repository: cd into the correct folder, or initialize the intended practice folder.

02 Remote origin already exists: inspect git remote -v; use git remote set-url origin YOUR_REPOSITORY_URL if necessary.

03 Push rejected: check whether the remote has commits you do not have; inspect before pulling.

04 Authentication failed: confirm account access and use the supported credential flow.

05 Accidentally exposed a key: revoke/rotate it. Deleting the visible file does not erase its history.

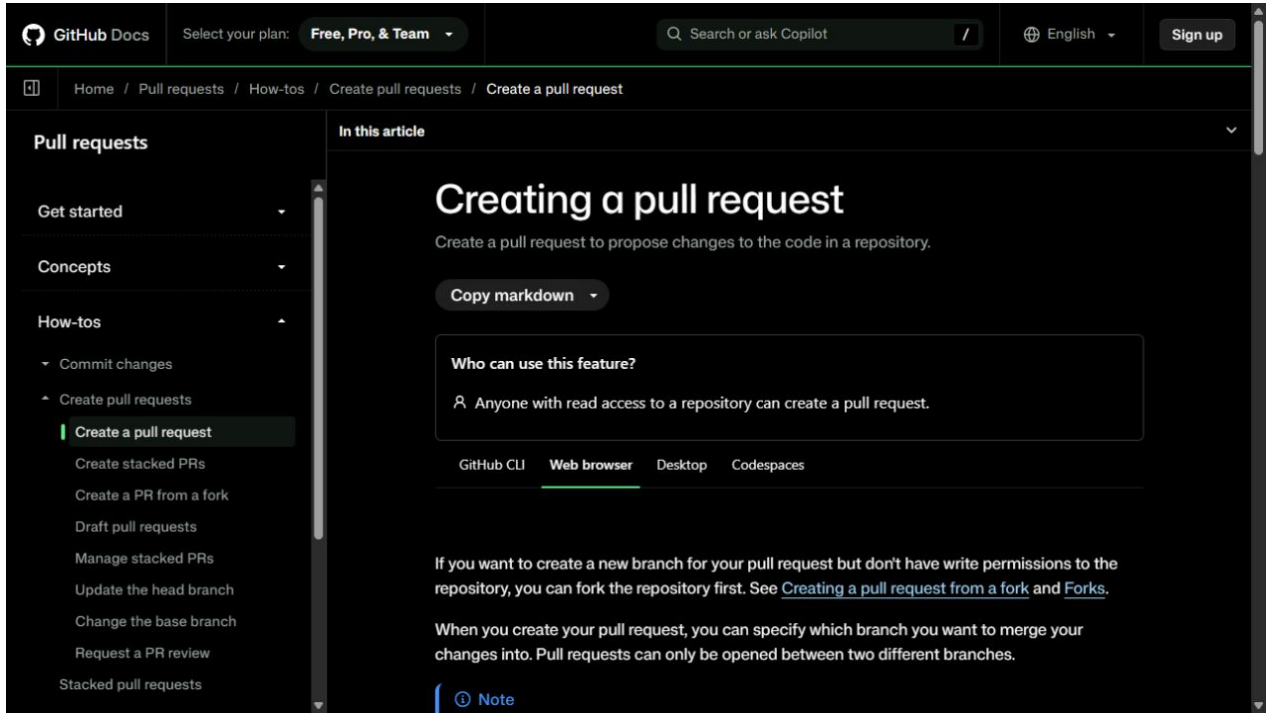
Advanced section: feature branches

1. Begin on an updated main branch with a clean working tree.
2. Create a feature branch and change hello.py.
3. Run the file, review the diff and commit the change.
4. Push the branch. The remote branch now contains your proposed change.

```
git switch main
git pull --ff-only
git switch -c feature/greeting
git add hello.py
git commit -m "Improve greeting"
git push -u origin feature/greeting
```

Advanced section: pull request and merge

1. On GitHub, open Compare & pull request for feature/greeting.
2. Confirm base=main and compare=feature/greeting.
3. Explain the change, review Files changed and verify the example runs.
4. Create the pull request, then merge it when the change is ready.
5. Locally switch to main and pull. You may delete the merged local branch afterward.



Official screen reference; interface labels may change.

Official reference

Advanced section: local cleanup

Refresh main after the GitHub merge.

If git branch -d refuses deletion after a squash merge, keep the branch until you verify its changes are present.

Checkpoint: demonstrate two commits on GitHub and explain the edit/stage/commit/push cycle.

Advanced checkpoint: demonstrate a merged pull request and an updated local main.

```
git switch main
git pull --ff-only
git branch -d feature/greeting
git status
```